MEDIA FORMATION
GAGNER VOTRE VIE PROFESSIONNELLE

Intitulé du cours: JavaScript

Chapitre 3: le DOM JAVASCRIPT

Filière: 2éme année – BTS-MARKETING & MULTIMEDIA

JavaScript

Enseignante: Abid Arij

Année de formation 2025-2026

Objectifs du module

À la fin du module, l'apprenant doit être capable de :

- Identifier le DOM et sa structure
  - Comprendre les nœuds, les éléments, et l’arborescence de la page.
- Accéder aux éléments HTML
    - getElementById
    - getElementsByClassName
    - getElementsByTagName
    - querySelector / querySelectorAll
- Modifier le contenu et l’apparence
  - Modifier le texte avec innerHTML
  - Changer le style avec .style
  - Ajouter ou supprimer des classes avec classList
- Créer des éléments dynamiques
  - Ajouter, supprimer ou modifier des éléments HTML via JavaScript.

<script type="text/javascript">
switch (new Date().getDay()) {
case 6:
text = "Friday";
break;
case 0:
text = "Sunday";
break;
default:
text = "Choose Your Day";
}
</script>
Javascript

01

le DOM

Qu’est-ce que le DOM ?

JavaScript

Le **DOM (Document Object Model)** est une représentation en mémoire de la page HTML.
Le navigateur transforme le HTML en une **arborescence d’objets** (nœuds) que JavaScript peut lire et modifier.

- Le **document** racine représente la page.
- Chaque balise HTML devient un **Element node**.
- Le DOM permet de lire/modifier la structure, le contenu et les styles dynamiquement.

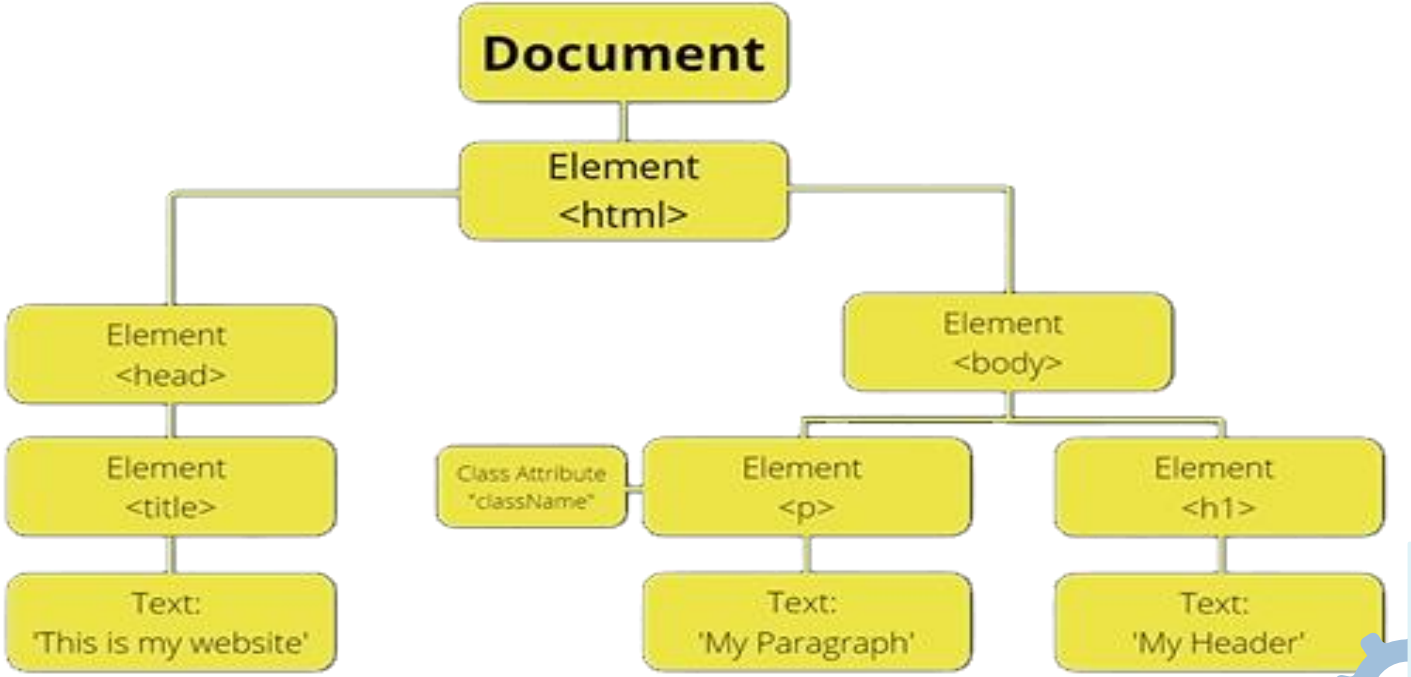
Pourquoi on utilise le DOM ?

Le DOM est utilisé pour rendre la page **interactives et dynamiques**. Sans le DOM, une page web serait **statique**, c'est-à-dire juste du texte et des images sans aucune interaction.

Exemples :

- **Modifier du texte ou du contenu**
- **Changer le style ou l'apparence**
- **Réagir aux actions de l'utilisateur**

DOM Tree


Document
Element
<html>
Element
<head>
Element
<body>
Element
<title>
Class Attribute
"className"
Element
<p>
Element
<h1>
Text:
'This is my website'
Text:
'My Paragraph'
Text:
'My Header'

Quand tu écris une page HTML :

```
<h1>Bienvenue</h1>
<p id="msg">Bonjour</p>
```

Le navigateur transforme ça en **arbre DOM** :

```
Document
└── html
    ├── head
    └── body
        ├── h1
        └── p id="msg"
```

→ JavaScript peut alors accéder aux éléments et les **modifier en direct** (sans recharger la page).

Concepts de base

- **Window** : window est l'**objet global** qui représente la **fenêtre ou l'onglet du navigateur**.

Tout ce qui se passe dans le navigateur (alert, timers, DOM, stockage local, etc.) est relié à window.

- **document** : racine du DOM (window.document).

document représente **tout le contenu HTML de la page**.
C'est à partir de document que tu peux accéder aux éléments, créer du contenu, modifier le texte, etc.

Concepts de base

- **Node :** Un Node (nœud) est **tout type de “pièce” du DOM**.
- Types de nœuds :
  - **Element** → une balise HTML (<p>, , <ul>…)
  - **Text** → le texte à l’intérieur d’un élément
  - **Comment** → un commentaire HTML (<!-- commentaire -->)

- **Element**
- Un Element est **un Node particulier qui représente une balise HTML**

- **Tree – L’arbre DOM**
- La structure DOM est comme **un arbre** :
  - **Parent** : l’élément qui contient d’autres éléments
  - **Children** : les éléments contenus par le parent
  - **Siblings** : les éléments “frères/sœurs” au même niveau

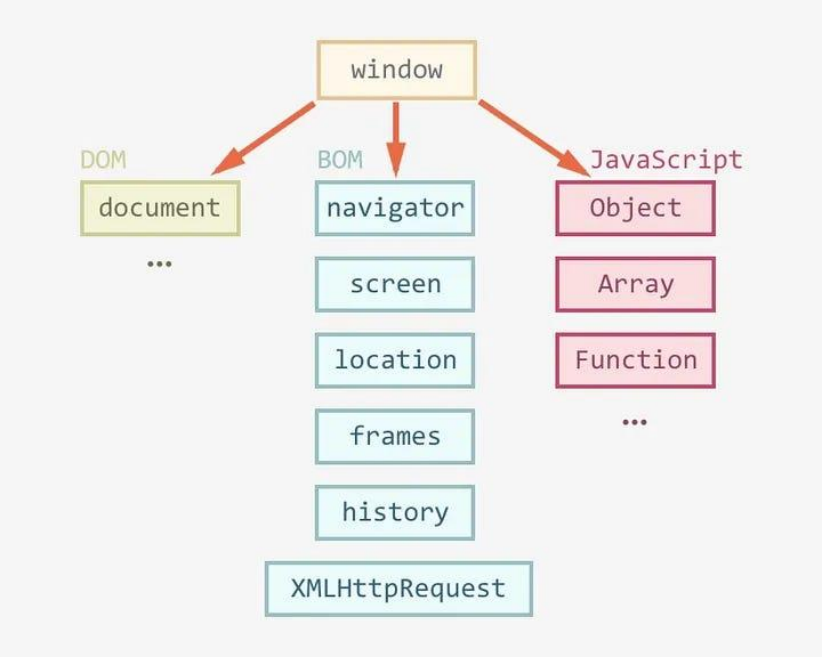
Différence entre window et document en JavaScript (DOM)

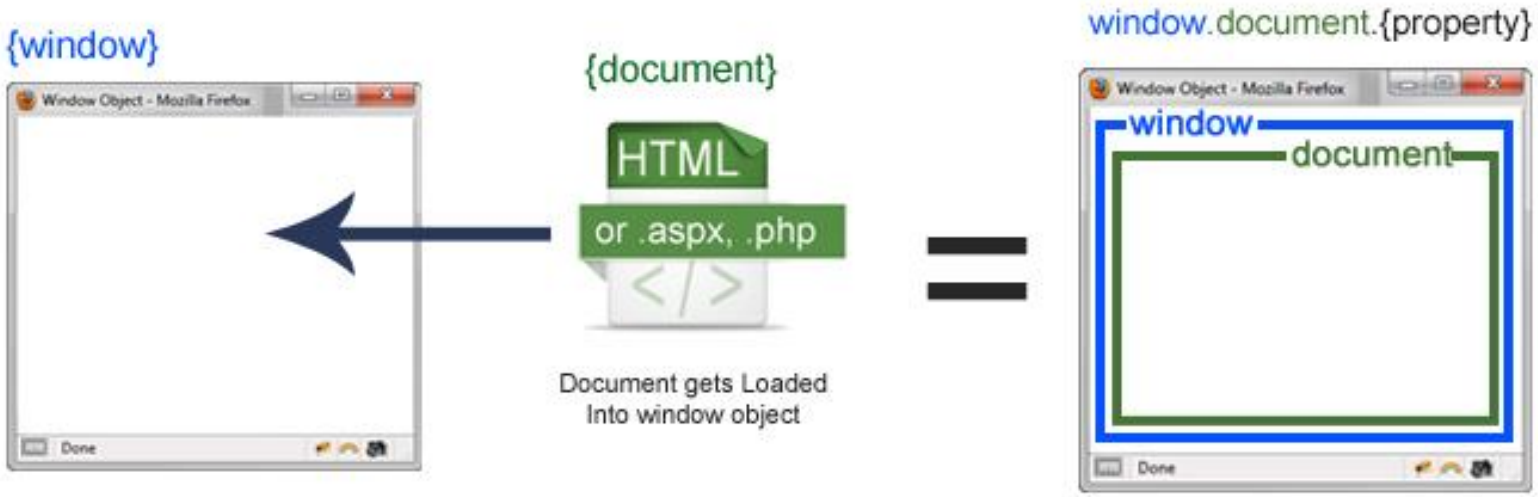
❖ **window**

window représente **la fenêtre du navigateur**.
C'est **l'objet global dans le navigateur.**
Il contient tout : le document, les fonctions globales (alert, setTimeout, etc.), les variables globales et les objets du navigateur.

❖ **document**

document représente **le contenu de la page HTML**.
Il nous permet de **manipuler le contenu de la page** (textes, images, balises, attributs…).
C'est une partie de window.




{window}
Window Object - Mozilla Firefox
Done
{document}
HTML
or .aspx, .php
Document gets Loaded
Into window object
window.document.{property}
Window Object - Mozilla Firefox
window
document
Done

Les principales méthodes d'accès au DOM

- Quand une page web est chargée, le navigateur transforme ton code HTML en une **structure appelée DOM** (Document Object Model).

- Grâce au **DOM**, JavaScript peut **retrouver un élément HTML** (comme un <p>, un , un <button>) et le **manipuler** : changer son texte, sa couleur, le cacher, etc.

On peut accéder par:

- ❑ **Accès par attribut id**
- ❑ **Accès par attribut name**
- ❑ **Accès par nom de balise ou nom de classe**
- ❑ **Accès par sélecteur CSS**

Accès aux éléments HTML : par « id »
a) document.getElementById('id')

Sélectionne **un seul élément** par son **id** (unique sur la page).

```
<h1 id="titre">Bonjour le DOM</h1>

<script>

const titre = document.getElementById("titre");
console.log(titre);
titre.style.backgroundColor="pink";

</script>
```

Exemple

```
<h1 id="title">Titre original</h1>
<button onclick="changerTitre()">Changer le titre</button>

<p id="resultat"></p>
```

```
<script>
function changerTitre()
 {
 let titre = document.getElementById("title");

titre.innerHTML = "Nouveau titre modifié !";

document.getElementById("resultat").innerHTML =
        "Le titre a été modifié avec getElementById()";
}
</script>
```

innerHTML : c'est quoi ?

innerHTML est une propriété en JavaScript qui permet de **lire ou modifier le contenu HTML à l'intérieur d'un élément**.

Autrement dit, ça sert à **changer ce qu'il y a dedans** un élément HTML (texte, balises, images…).

1) Afficher du texte dans la page

```
<p id="message"></p>

<script>
document.getElementById("message").innerHTML = "Bonjour !";
</script>
```

➞ **Le texte “Bonjour !” apparaît dans la page.**

2) Modifier du texte existant

```
<h1 id="titre">Titre original</h1>

<script>
document.getElementById("titre").innerHTML = "Titre modifié par JavaScript";
</script>
```

➞ Le contenu du <h1> change directement.

3) Ajouter du HTML (pas seulement du texte) : Tu peux insérer des **balises** dedans.

Exemple :

```
<div id="zone"></div>

<script>
document.getElementById("zone").innerHTML = "<strong>Texte en gras</strong>";
</script>
```

➞ Le texte apparaît en **gras** parce que tu as inséré une balise .

4) Afficher un résultat après un calcul ou un traitement

```
<p id="resultat"></p>
<button onclick="calcul()">Tester</button>

<script>
function calcul() {
    let x = 10 + 5;
    document.getElementById("resultat").innerHTML =
        "Le résultat est : " + x;
}
</script>
```

Accès aux éléments HTML : par « name»
b) document.getElementsByName('nom')

C'est quoi getElementsByName() ?
Cette méthode permet d'accéder **à tous les éléments HTML** qui ont un **attribut name** donné.

```
document.getElementsByName("nom");
```

document → la page HTML entière
getElementsByName() → signifie "récupère les éléments qui ont le nom…"
"nom" → correspond à la valeur de l'attribut name dans ton code HTML

Accès aux éléments HTML : par « name»
b) document.getElementsByName('nom')

Retourne un tableau d'objets HTML ayant nom défini dans la propriété « name» de la balise de l'objet (l'attribut name est utlisé avec les objets formulaire et déprécié pour tout autre éléments).

```
<form name="form_fruit">

  <label>fruit </label>
  <input type="checkbox" name="fruit" value="fraise">fraise <br/>
  <input type="checkbox" name="fruit" value="banane">banane <br/>
  <input type="checkbox" name="fruit" value="pomme">pomme<br/>
</form>
<script>
document.getElementsByName("fruit")[1].checked = true;

</script>
```

Explication de l'exemple :

document.getElementsByName("fruit")

- Récupère **tous les éléments HTML dont l'attribut name est "fruit"**.
- Ici, cela retourne **une collection de 3 cases à cocher** : [fraise, banane, pomme]
- .[1] Sélectionne **le deuxième élément** de la collection (en JS, l'index commence à 0).
  Donc [1] correspond à la **case "banane"**.
- .checked = true ⇨ Définit l'état de la case à cocher.
- true → coche la case.
- false → décoche la case.

Accès aux éléments HTML : par « classe»
c) document.getElementsByClassName('classe')

Cette méthode permet de **récupérer tous les éléments ayant une certaine classe**.
Elle retourne une **collection d'éléments** (comme un tableau).
Ensuite, tu peux **modifier leur texte, style ou comportement** via JavaScript.

```
<div class="l1"> bonjour </div>
    <span class="l1"> bonsoir </span>
    <p class="l1"> salut </p>

    <script>
        let elements = document.getElementsByClassName('l1');

        for (let i = 0; i < elements.length; i++) {
            alert('Élément n° ' + (i + 1) + ' : ' + elements[i].innerText);
        }
    </script>
```

Accès aux éléments HTML : par « classe»
c) document.getElementsByClassName(‘classe’)

Explication de l’exemple :

- **document.getElementsByClassName('l1’)**

C’est une **méthode du DOM** (Document Object Model) qui **récupère tous les éléments HTML ayant la classe 'l1’**.
Elle renvoie une **collection (comme un tableau)** d’éléments.
Ici, elements contiendra 3 éléments : le , le et le <p>.

- **La boucle for:** for (let i = 0; i < elements.length; i++) { }

Parcourt tous les éléments récupérés (ici 3).

- i est l’index de l’élément actuel dans la collection.
- **alert('Élément n° ' + (i + 1) + ' : ' + elements[i].innerText);**

Affiche une **fenêtre d’alerte** pour chaque élément.

- **i + 1** : sert juste à numéroter les éléments de **1 à n (au lieu de 0 à n-1).**
- **elements[i].innerText** : récupère le **texte à l’intérieur de l’élément HTML** (ici : "bonjour", "bonsoir", "salut").

Accès aux éléments HTML : par nom de balise
d) document. getElementsByTagName('balise')

récupérer tous les éléments HTML qui ont **un certain type de balise**, par exemple ** , <p>, , <h1>,** etc.
Ce qu'il renvoie : une **collection HTML** (ressemblant à un tableau) contenant tous les éléments correspondants.

Accès aux éléments HTML : par nom de balise
d) document. getElementsByTagName('balise')

```
<div> bonjour </div>
    <div> bonsoir </div>

    <script>
        let divs = document.getElementsByTagName('div');

        for (let i = 0; i < divs.length; i++) {
            alert('Élément n° ' + (i + 1) + ' : ' + divs[i].innerText);
        }
    </script>
```

- La boucle for Parcourt **tous les éléments de la collection divs**.
- i est l'index, qui commence à 0 et va jusqu'au nombre d'éléments - 1.
- alert('Élément n° ' + (i + 1) + ' : ' + divs[i].innerText);
- i + 1 → sert à numéroter les éléments à partir de 1 (plutôt que 0).
- divs[i].innerText → récupère le **texte à l'intérieur du **.

Accès aux éléments HTML : par Sélector
e.Accès aux éléments HTML : par **querySelector()**

Permet de récupérer **un seul élément** (le premier trouvé).

On peut utiliser :

- un ID → #monID
- une classe → .maClasse
- une balise → p
- un sélecteur CSS combiné → div p.texte

Ensuite, on peut faire des modifications avec **.innerHTML, .style,** etc.

❖ Sélectionner par id

document.querySelector("#monTitre");

```
<h1 id="monTitre">Titre original</h1>
<button onclick="changerID()">Changer le titre</button>
<p id="msgID"></p>

<script>
function changerID() {
   let titre = document.querySelector("#monTitre");
   titre.innerHTML = "Titre modifié avec querySelector par ID";
   document.getElementById("msgID").innerHTML = "ID sélectionné avec #";
}
</script>
```

#monTitre sélectionne le **élément avec l'ID monTitre**.

❖ Sélectionner par classe

document.querySelector(".paragraphe");

```
<p class="paragraphe">Paragraphe 1</p>
<p class="paragraphe">Paragraphe 2</p>
<button onclick="changerClasse()">Changer le premier paragraphe</button>
<p id="msgClasse"></p>

<script>
function changerClasse() {
   let p = document.querySelector(".paragraphe");
   p.style.color = "blue";
   document.getElementById("msgClasse").innerHTML = "La première classe sélectionnée
est devenue bleue";
}
</script>
```

`.paragraphe` sélectionne **le premier élément avec la classe `paragraphe`**.

❖ Sélectionner par balise

document.querySelector("p");

```
<p>Premier paragraphe</p>
<p>Deuxième paragraphe</p>
<button onclick="changerBalise()">Changer le premier paragraphe</button>
<p id="msgBalise"></p>

<script>
function changerBalise() {
   let p = document.querySelector("p");
   p.style.fontWeight = "bold";
   document.getElementById("msgBalise").innerHTML = "Le premier paragraphe est en gras";
}
</script>
```

"p" sélectionne **le premier <p> dans la page**.

❖ Sélecteur CSS combiné

On peut combiner balises, classes, ID, ou descendants.

```
<div>
   <p class="texte">Paragraphe 1 dans div</p>
   <p class="texte">Paragraphe 2 dans div</p>
</div>
<p class="texte">Paragraphe hors div</p>

<button onclick="changerCombiné()">Modifier le premier paragraphe </button>
<p id="msgCombiné"></p>

<script>
function changerCombiné() {
   let p = document.querySelector("div p.texte");
   p.style.backgroundColor = "yellow";
   document.getElementById("msgCombiné").innerHTML = "fond jaune";
}
</script>
```

`div p.texte` sélectionne **le premier `<p>` avec la classe `texte`**

Sélecteur CSS	Exemple querySelector	Résultat
ID	#monID	Sélectionne l’élément avec cet ID
Classe	.maClasse	Sélectionne le premier élément avec cette classe
Balise	p	Sélectionne le premier <p>
Combiné	div p.texte	Premier <p class="texte"> dans un

```
<h1 class="titre">Bonjour tout le monde</h1>

<p id="message">Ceci est un paragraphe.</p>

<button id="btn">Changer le texte</button>

<script src="script.js"></script>
```

```
const titre = document.querySelector(".titre");

const message = document.querySelector("#message");

const bouton = document.querySelector("button");


bouton.addEventListener("click", function () {
    titre.textContent = "Titre modifié avec querySelector() !";
    message.textContent = "Le texte du paragraphe a changé !";

    // Changer la couleur
    titre.style.color = "blue";
    message.style.color = "green";
});
```

À quoi sert addEventListener() en JavaScript ?

addEventListener() sert à **écouter** et **réagir** aux actions de l'utilisateur sur une page web.
Un événement = une action
Exemples :

- clic sur un bouton
- survol de la souris
- saisir du texte
- appuyer sur une touche
- charger la page
- faire défiler (scroll)

❖

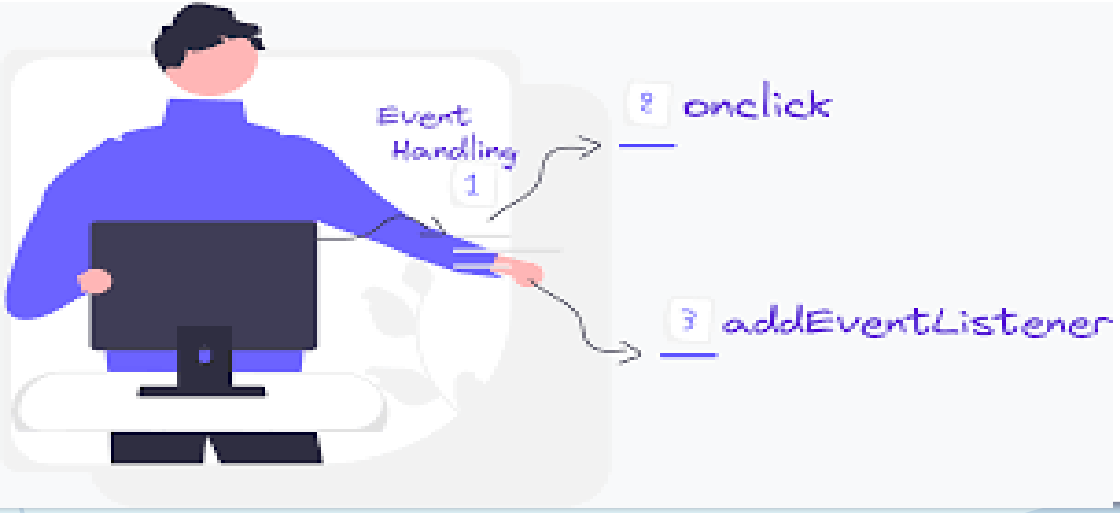
Son rôle :

addEventListener() permet de dire au navigateur :
"Quand ceci se produit, exécute ce code."

Syntaxe :

```
element.addEventListener("type_evenement", fonction);
```

- **type_evenement** → **"click", "mouseover", "keyup", "submit", etc.**
- **fonction** → ce qui doit se passer quand l'événement arrive.



❖ Sélectionne tous les éléments querySelectorAll()

querySelectorAll() permet de **sélectionner tous les éléments** qui correspondent à un **sélecteur CSS**.

✓ Tous les paragraphes
✓ Tous les éléments d'une classe
✓ Tous les éléments d'un type
✓ Tous les éléments dans un div
✓ Tous les boutons, images, titres…

Sélectionne tous les éléments querySelectorAll()

- **Exemple:**

```
<p class="color">Texte 1</p>
<p class="color">Texte 2</p>
<p class="color">Texte 3</p>
<button id="btnColor">Changer couleur</button>
```

```
document.getElementById("btnColor").onclick = function() {
    let textes = document.querySelectorAll(".color");

textes[0].style.color = "red";
    textes[1].style.color = "green";
    textes[2].style.color = "blue";
};
```

En JavaScript moderne, on utilise surtout :
querySelector() → sélectionne **le premier élément** qui correspond au sélecteur.
querySelectorAll() → sélectionne **tous les éléments** qui correspondent au sélecteur (retourne une NodeList, comme un tableau).

Méthode	Sélectionne	Exemple
getElementById	1 élément	#id
getElementsByClassName	plusieurs	.classe
getElementsByTagName	plusieurs	p
querySelector	1 élément (le premier)	CSS
querySelectorAll	plusieurs éléments	CSS

🌿 Manipulation des attributs HTML

Les éléments HTML possèdent des **attributs**.

```
<img src="image.jpg" alt="photo">
<input type="text" placeholder="Votre nom">
<a href="https://google.com">Google</a>
```

Avec JavaScript, on peut :

- ✓ Lire un attribut
- ✓ Modifier un attribut
- ✓ Ajouter un attribut
- ✓ Supprimer un attribut

Méthodes principales pour manipuler les attributs

Il y a **3 méthodes fondamentales** :

⭐ **1) getAttribute()**
Lire un attribut

⭐ **2) setAttribute()**
Modifier / ajouter un attribut

⭐ **3) removeAttribute()**
Supprimer un attribut

1. Accéder à un attribut

Tu dois d'abord sélectionner l'élément :

```
<img id="photo" src="image1.jpg" alt="Image 1">
<button id="btn">Afficher le src</button>
<p id="resultat"></p>
```

```
document.getElementById("btn").onclick = function() {

    let img = document.getElementById("photo");

    let valeur = img.getAttribute("src");

    document.getElementById("resultat").textContent = "Le src est : " + valeur;
};
```

⇨ getAttribute("src") permet de **lire** l'attribut.

2. Modifier un attribut

```
<img id="photo" src="image1.jpg" alt="Image">
<button id="change">Changer d'image</button>
```

```
document.getElementById("change").onclick = function() {
    let img = document.getElementById("photo");

    img.setAttribute("src", "image2.jpg");
};
```

setAttribute() sert à **modifier** la valeur d'un attribut.

3. Ajouter un attribut

On peut ajouter n'importe quel attribut HTML.

HTML :

```
<p id="txt">Bonjour</p>
<button id="ajouter">Ajouter l'attribut title</button>
```

```
document.getElementById("ajouter").onclick = function() {
    let p = document.getElementById("txt");

    p.setAttribute("title", "Ceci est un tooltip");
};
```

L'attribut title apparaît au survol de la souris.

4. Supprimer un attribut

```
<input id="champ"  type="text" placeholder="Écris ici">
<button id="supprimer">Supprimer placeholder</button>
```

```
document.getElementById("supprimer").onclick = function() {
    let input = document.getElementById("champ");

    input.removeAttribute("placeholder");
};
```

→ removeAttribute() supprime totalement l'attribut.

5. Manipuler les attributs les plus utilisés

Les éléments du formulaire.

Elément(s)	Propriétés JS
Ligne de text, textarea, mot de passe	Name Value defaultValue
checkbox	Name Value Checked defaultChecked
radio	Name Value Checked defaultChecked Index(le rang)
Select (liste de sélection)	Name length defaultselected selecteIndex
Buttons (submit,reset,button)	Name value

Exemple:

```
<h3>Exemple getAttribute avec href</h3>

<a id="monLien" href="https://google.com">Aller sur Google</a>
<button id="btnAfficher">Afficher le lien</button>

<p id="resultatLien"></p>
```

```
document.getElementById("btnAfficher").onclick = function() {
    // Sélectionner le lien
    let lien = document.getElementById("monLien");

    // Récupérer la valeur de l'attribut href
    let url = lien.getAttribute("href");

    // Afficher la valeur dans la page
    document.getElementById("resultatLien").textContent =
        "L'adresse du lien est : " + url;
};
```

🌿 Manipulation du contenu des éléments HTML

Méthodes principales pour manipuler le contenu

- En JavaScript, pour **lire ou modifier le contenu**, on utilise :

Méthode / Propriété	Usage	Exemple
textContent	Lire ou modifier le **texte brut**	element.textContent = "Nouveau texte"
innerHTML	Lire ou modifier le **HTML interne**	element.innerHTML = "Texte en gras"
innerText	Similaire à textContent, mais ignore le CSS display:none	element.innerText = "Texte visible"

Manipulation des éléments HTML: contenu

textContent

- Lit ou modifie le **texte à l'intérieur d'un élément HTML**.
- **Ignore complètement les balises HTML.**

```
<p id="para1">Bonjour tout le monde !</p>
<button id="btn1">Changer le texte</button>
```

```
document.getElementById("btn1").onclick = function() {
    let p = document.getElementById("para1");
    p.textContent = "Le texte a été modifié avec textContent !";
};
```

Résultat : Le texte du paragraphe change **mais aucune balise HTML ne peut être ajoutée ici**.

Manipulation des éléments HTML: contenu

innerHTML

- Permet de lire ou modifier le **contenu HTML** (texte + balises).
- Utile si tu veux ajouter du **formatage**, des liens ou des images.

- **Exemple:**

```
<div id="zoneHTML">Texte normal</div>
<button id="btnHTML">Ajouter du HTML</button>
```

```
document.getElementById("btnHTML").onclick = function() {
    let zone = document.getElementById("zoneHTML");
    zone.innerHTML = "<b>Texte en gras</b> et <i>italique</i>";
};
```

Résultat : le texte devient **gras et italique**, directement visible dans la page.

Manipulation des éléments HTML: contenu innerText

- Lit le texte **que l'utilisateur peut voir à l'écran**
- Ignore le texte caché par CSS (display:none) ou invisible
- **Exemple**

```
<p id="p1">Visible <span style="display:none">Invisible</span></p>
<button id="btnInnerText">Lire innerText</button>
<p id="result1"></p>
```

```
document.getElementById("btnInnerText").onclick = function() {
    let texte = document.getElementById("p1").innerText;
    document.getElementById("result1").textContent = texte;
};
```

Résultat :
Affiche : **Visible**
Le texte **"Invisible"** n'apparaît pas car il est caché par le CSS.

Manipulation du style CSS

Avec JavaScript, tu peux **modifier dynamiquement l'apparence d'un élément** : couleur, taille, bordure, affichage, etc.
Il existe plusieurs façons :

- Utiliser la propriété .style
- Ajouter / retirer des classes CSS avec .classList

1 Modifier directement un style avec `.style`

- **Syntaxe:** `element.style.proprieteCSS = "valeur";`

•**Exemple :**

```
div.style.backgroundColor = "red";
div.style.fontSize = "20px";
```

Manipulation du style CSS

```
<p id="txt1">Bonjour !</p>
<button id="btnStyle">Changer style</button>
```

```
document.getElementById("btnStyle").onclick = function() {
    let p = document.getElementById("txt1");

    p.style.color = "white";            // couleur du texte
    p.style.backgroundColor = "blue";   // fond
    p.style.fontSize = "24px";          // taille du texte
    p.style.padding = "10px";           // marge intérieure
};
```

Manipulation du style CSS

Modifier plusieurs styles à la fois avec cssText

- **Syntaxe:** element.style.cssText = "propriete1: valeur1; propriete2: valeur2;";
- **Exemple:**

```
<div id="box">Box</div>
<button id="btnCssText">Modifier styles</button>
```

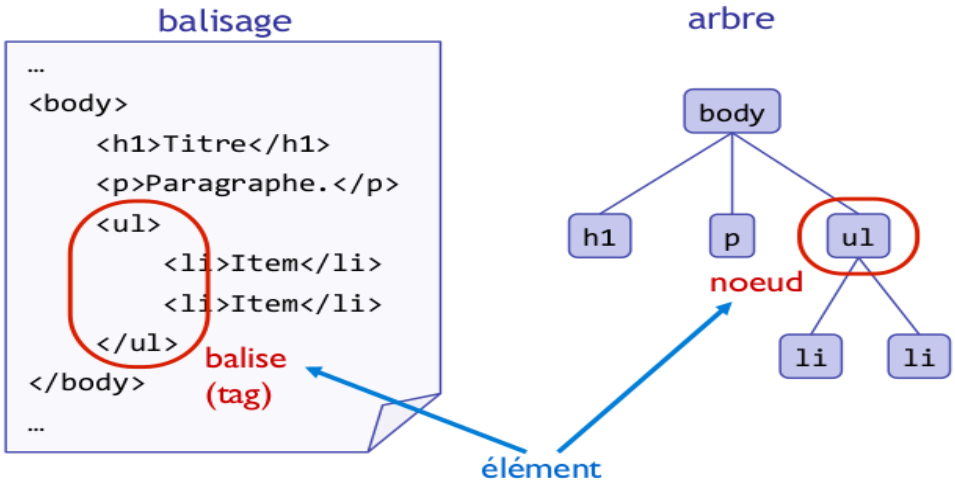
```
document.getElementById("btnCssText").onclick = function() {
    let box = document.getElementById("box");
    box.style.cssText = "background-color: orange; color: white; border: 2px solid black; padding: 15px;";
};
```

Résultat :
Le div devient orange avec texte blanc, bordure et padding.

🌿 Modification de l’arbre DOM

Modification de l'arbre DOM et création de nouveaux éléments

- Le **DOM (Document Object Model)** représente la structure HTML de la page comme un **arbre d'objets**.
- Chaque balise HTML est un **nœud** dans cet arbre.
  Grâce à JavaScript, tu peux **modifier cet arbre** : ajouter, supprimer ou modifier des éléments.



🌿 Création et insertion d'un nouvel élément HTML

- En JavaScript, il est possible de **créer de nouveaux éléments** (paragraphe, div, image, bouton, etc.) et de les **ajouter dynamiquement** à la page.

Méthodes principales:

- **document.createElement(tag)** → Crée un nouvel élément HTML avec la balise tag.
- **element.textContent ou element.innerHTML** → Définir le contenu du nouvel élément.
- **parent.appendChild(element)** → Ajouter l'élément dans le DOM à un élément parent.
- **parent.insertBefore(newEl, refEl)** → Ajouter l'élément avant un autre élément existant

🌿 Création et insertion d'un nouvel élément HTML

- Avec DOM, l'ajout d'un élément HTML se fait en trois étapes

1 Étapes pour créer un élément

Créer l'élément :

```
let element = document.createElement("tag");
```

2 Ajouter du contenu ou des attributs :

Texte : element.textContent = "Texte"

Classe : element.classList.add("maClasse")

Attribut : element.setAttribute("src", "image.jpg")

3 Ajouter l'élément à la page :

```
parent.appendChild(element);
```

parent = l'élément dans lequel tu veux ajouter le nouveau contenu (ex : , <ul>).

Ajouter un élément – appendChild()

- Ajoute un **élément enfant à la fin** d'un autre élément.
- Généralement utilisé après createElement().

Exemple:

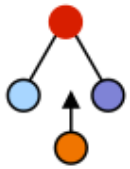
```
<ul id="liste">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
<button id="btnAjouter">Ajouter un item</button>
```

```
document.getElementById("btnAjouter").onclick = function() {
    let li = document.createElement("li");
    li.textContent = "Item ajouté";

    document.getElementById("liste").appendChild(li);
};
```

Résultat : Un nouvel <li> est ajouté **à la fin de la liste**.

Insérer un élément avant un autre – insertBefore()



insertBefore

Permet d'insérer un **nouvel élément avant un élément existant**.

- **Syntaxe :**

parent.insertBefore(nouvelElement, elementDeReference);

- **Exemple:**

```
<ul id="liste2">
  <li id="i1">café</li>
  <li id="i2">jus</li>
</ul>
<button id="btnAvant">Ajouter avant jus</button>
```

```
document.getElementById("btnAvant").onclick = function() {
    let li = document.createElement("li");
    li.textContent = "milkshake";

    let parent = document.getElementById("liste2");
    let reference = document.getElementById("i2");

    parent.insertBefore(li, reference);
};
```

Supprimer un élément – removeChild()

removeChild

Supprime un **élément enfant** d'un élément parent.

- **Syntaxe :** parent.removeChild(enfant);
- **Exemple:**

```
<ul id="liste3">
  <li id="item1">Item 1</li>
  <li id="item2">Item 2</li>
</ul>
<button id="btnSupprimer">Supprimer Item 1</button>
```

```
document.getElementById("btnSupprimer").onclick = function() {
    let parent = document.getElementById("liste3");
    let enfant = document.getElementById("item1");

    parent.removeChild(enfant);
};
```

🌿 createTextNode() – Ajouter du texte dans le DOM

- document.createTextNode() permet de **créer un nœud texte**.

Utile lorsque tu veux **ajouter du texte à un élément** sans utiliser innerHTML (sécurisé contre le HTML injecté).

- **Syntaxe :**
- let texte = document.createTextNode("Mon texte ici");

Ensuite, on peut **attacher ce texte à un élément parent** avec appendChild ou insertBefore.

🌿 createTextNode() – Ajouter du texte dans le DOM

Exemple:

```
<div id="zoneTexte"></div>
<button id="btnTexte">Ajouter texte</button>
```

```
document.getElementById("btnTexte").onclick = function() {
    let div = document.getElementById("zoneTexte");

    // Créer un nœud texte
    let texte = document.createTextNode("Bonjour ! Je suis un texte créé avec createTextNode.");

    // Ajouter le texte dans le div
    div.appendChild(texte);
};
```

Résultat :
Le texte apparaît dans le lorsque tu cliques sur le bouton.

🌿 Navigation dans le DOM

🌿 Navigation dans le DOM

Le **DOM** est un arbre d'éléments HTML. Naviguer dans le DOM permet de **se déplacer entre les éléments** : parent, enfants, frères/sœurs, etc.
C'est essentiel pour manipuler la page dynamiquement.

Accéder aux enfants – children et childNodes

1. children

Retourne **uniquement les éléments HTML enfants** (ignore les textes et commentaires).

- **Syntaxe :** parent.children[index]
- **Exemple:**

```
<ul id="liste">
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
</ul>
<button id="btnEnfants">Afficher le premier enfant</button>
<p id="resEnfants"></p>
```

```
document.getElementById("btnEnfants").onclick = function() {
    let ul = document.getElementById("liste");
    let premierEnfant = ul.children[0]; // premier <li>
    document.getElementById("resEnfants").textContent =
        "Premier enfant : " + premierEnfant.textContent;
};
```

childNodes

childNodes retourne **tous les nœuds enfants** d'un élément, **y compris** :

- éléments HTML (<p>, <li>, etc.)
- textes (espaces, retours à la ligne)
- commentaires HTML
- Contrairement à children, childNodes **ne filtre pas les nœuds texte**

•Chaque nœud peut être **accédé avec un index** :

```
parent.childNodes[0]
parent.childNodes[1]
```

childNodes

```javascript
document.getElementById("btnChild").onclick = function() {
    let div = document.getElementById("maDiv");
    let res = "";

    // Accéder individuellement à chaque childNodes
    let node0 = div.childNodes[0]; // nœud texte "Texte avant"
    let node1 = div.childNodes[1]; // <p>Paragraphe 1</p>
    let node2 = div.childNodes[2]; // <p>Paragraphe 2</p>
    let node3 = div.childNodes[3]; // nœud texte "Texte après"

    // Construire le résultat
    res += "Node 0 : " + node0.nodeValue + "<br>";
    res += "Node 1 : " + node1.textContent + "<br>";
    res += "Node 2 : " + node2.textContent + "<br>";
    res += "Node 3 : " + node3.nodeValue + "<br>";

document.getElementById("resChild").innerHTML = res;
};
```

```html
<div id="maDiv">
    Texte avant
    <p>Paragraphe 1</p>
    <p>Paragraphe 2</p>
    Texte après
</div>

<button id="btnChild">
     Afficher nœuds enfants

</button>
<p id="resChild"></p>
```

Accéder au parent – parentNode et parentElement

1. parentNode

Retourne le **nœud parent** de l'élément.

- **Syntaxe :** let parent = element.parentNode;
- **Exemple:**

```
<div id="conteneur">
    <p id="para">Texte du paragraphe</p>
</div>
<button id="btnParent">Afficher parent</button>
<p id="resParent"></p>
```

```
document.getElementById("btnParent").onclick = function() {
    let para = document.getElementById("para");
    let parent = para.parentNode;
    document.getElementById("resParent").textContent =   "Parent de l'élément : " + parent.id;
};
```

Résultat : Parent de l'élément : conteneur

Accéder aux frères/sœurs – nextElementSibling et previousElementSibling

1. nextElementSibling : Retourne **l'élément suivant** (ignore les textes et commentaires).

2. previousElementSibling : Retourne **l'élément précédent**.

- **Exemple:**

```
<ul>
    <li id="i1">Item 1</li>
    <li id="i2">Item 2</li>
    <li id="i3">Item 3</li>
</ul>
<button id="btnFrere">Afficher frère suivant de Item 1</button>
<p id="resFrere"></p>
```

```
document.getElementById("btnFrere").onclick = function() {
    let i1 = document.getElementById("i1");
    let suivant = i1.nextElementSibling;
    document.getElementById("resFrere").textContent =
        "Frère suivant : " + suivant.textContent;
};
```

Accéder au premier et dernier enfant – firstElementChild, lastElementChild

- **firstElementChild** → Premier enfant élément
- **lastElementChild** → Dernier enfant élément
- **Exemple:**

HTML

```
<ul id="liste4">
    <li>Item A</li>
    <li>Item B</li>
    <li>Item C</li>
</ul>
<button id="btnFirstLast">
            Afficher premier
</button>
<p id="resFirstLast"></p>
```

JS

```
document.getElementById("btnFirstLast").onclick = function() {
    let ul = document.getElementById("liste4");
    let premier = ul.firstElementChild.textContent;
    let dernier = ul.lastElementChild.textContent;

    document.getElementById("resFirstLast").textContent =
        "Premier : " + premier + ", Dernier : " + dernier;
};
```

Résultat :

Premier : Item A, Dernier : Item C

Tableau résumé

Propriété / méthode	Description
children	Tous les éléments enfants
childNodes	Tous les nœuds enfants (texte + éléments)
firstElementChild	Premier enfant élément
lastElementChild	Dernier enfant élément
parentNode / parentElement	Élement parent
nextElementSibling	Frère suivant
previousElementSibling	Frère précédent

Merci pour votre attention